Quantifying quantumness: Determining the effect of quantum operations in machine learning models by comparison to classically simulatable variants

Víctor Mauricio Gutiérrez Luque

12, 05, 2026

Computations were performed with computing resources granted by RWTH Aachen University under project `thes2204`.

Abstract

Contents

Chapter 1

Background and Introduction

1.1 Quantum Computing Preliminaries

1.1.1 Behind Quantum Computing

In the last century, there were measured results in classical physics giving absurdities. Initially, when people studied how hot objects glowed, the old theory predicted they should emit infinite amounts of UV light. Planck resolved this problem by proposing that energy is not exchanged continuously, but in discrete packets called quanta (Planck, 1900). Subsequently, the photoelectric effect showed that light can knock electrons out of metal only if its frequency is high enough; brightness or intensity of light alone did not help. Einstein explained this phenomenon by saying light behaves like particles (photons), each carrying one energy packet (Einstein, 1905). A further development was Bohr's contribution. Atoms when heated were seen to emit only specific wavelengths or colors (spectral lines) and not a smooth rainbow. This made sense if electrons inside atoms can only have certain allowed energies. Bohr applied the quantum idea to atoms, proposing quantized energy levels for electrons in 1913, but without explaining why (Bohr, 1913). Consequently, De Broglie suggested that not only light, but also matter like electrons can act like waves; electron-diffraction experiments confirmed this (de Broglie, 1924).

Heisenberg and Schrödinger completed it by independently constructing new, self-consistent replacements for classical mechanics. Heisenberg started from the experimental fact that atomic spectra reveal only transitions between discrete energy levels. This led naturally to a matrix-based algebra in which the order of operations matters, capturing quantum behavior without relying on classical orbits. Schrödinger, in contrast, searched for a wave equation governing microscopic systems of matter. This yielded cor-

rect bound-state energies and spectral predictions. They were soon shown to be mathematically equivalent descriptions of the same theory, and Born provided the probabilistic interpretation of the wavefunction (Heisenberg, 1925; Schrödinger, 1926; Born, 1926). In short: classical physics failed for small-scale phenomena, and quantum mechanics was the new rulebook that fit the experiments.

In the classical computation model, information is encoded in bits and processed by deterministic or randomized operations, and the time and memory required by these operations determine what is computationally feasible. Quantum computing extends this model by introducing a new unit of information, the qubit, whose behavior follows the rules of quantum mechanics. Unlike a classical bit, which is always either 0 or 1, a qubit can occupy infinitely many states on a continuous state space; before measurement it can exist in a superposition of basis states; measurement irreversibly changes the qubit's state; and the outcome depends on the measurement basis, meaning the same qubit can be probed along many different axes (Nielsen and Chuang, 2010).

These features are valuable not because they allow unlimited extraction of information, but because they allow information to be encoded in complex amplitudes and phases, manipulated through continuous unitary transformations, and combined across qubits to produce interference and entanglement.

Well-designed quantum algorithms exploit these effects so that, after a sequence of gates, the probability of measuring a desired result is amplified while unwanted outcomes are suppressed. These capabilities have no direct classical analogue. This mechanism is behind landmark quantum speedups such as Shor's algorithm for integer factoring and discrete logarithms, which runs in polynomial time on a quantum computer compared to the best known sub-exponential classical methods (Shor, 1994, 1997), and Grover's algorithm for unstructured search, which reduces $O(N)$ classical query complexity to $O(\sqrt{N})$ using amplitude amplification (Grover, 1996).

We will now first introduce the minimal linear-algebra language needed to describe quantum states and operations; then define qubits and single-qubit gates; extend to multi-qubit systems via tensor products and entanglement; and finally present the circuit model as the formal framework for quantum computation.

1.1.2 Some Linear Algebra

We briefly review the basic linear algebra concepts used throughout this thesis.[1]

A vector is a geometric object that as a magnitude and a direction and can live in a certain vector space, represented as columns whose values are in the domain. The basic operations are vector addition and scalar multiplication, and any expression of the form $\sum_i \alpha_i v_i$ is a linear combination of vectors v_i with coefficients α_i.

A map between vector spaces is linear if it respects these operations, $L(\alpha v + \beta w) = \alpha L(v) + \beta L(w)$. The standard Hermitian inner product is $\langle v, w\rangle = \sum_i \overline{v_i} w_i$, which induces the norm $\|v\| = \sqrt{\langle v, v\rangle}$. Two vectors are orthogonal if $\langle v, w\rangle = 0$, and a vector is normalized if $\|v\| = 1$. We often write vectors and inner products in bra–ket notation, where a column vector is denoted $|v\rangle$, its conjugate transpose is $\langle v|$, and the inner product is $\langle v|w\rangle$.

A basis $\{e_1, \ldots, e_n\}$ of $\mathbb{C}^n$ is a set of vectors such that every vector has a unique coordinate representation $v = \sum_i \alpha_i e_i$; if, in addition, $\langle e_i, e_j\rangle = \delta_{ij}$, the basis is orthonormal. The standard (computational) basis of $\mathbb{C}^n$ consists of the unit vectors with a single 1 in one position and 0 elsewhere, and any vector can be written as a linear combination of these basis vectors.

Linear maps on the vector space are represented by matrices, and their action on a vector is given by matrix-vector multiplication; special directions that are only rescaled by a matrix are captured by eigenvectors v with eigenvalues λ, satisfying $Av = \lambda v$. Of particular importance are unitary matrices U, defined by the condition $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose and I is the identity. Unitary operators preserve inner products, and hence norms and angles, and are always reversible with inverse $U^{-1} = U^\dagger$.

1.1.3 Postulates of Quantum Mechanics

In this subsection we summarize the standard postulates of finite-dimensional quantum mechanics in the language of complex vector spaces introduced above. According more detailed discussion can be found in Nielsen and Chuang (2010, Chapter 2).

Postulate 1 (State space). To every isolated physical system there corresponds a complex Hilbert space $\mathcal{H}$, called the state space of the system. The (pure) states of the system are represented by unit vectors $|\psi\rangle \in \mathcal{H}$ with $\langle\psi|\psi\rangle = 1$. Two vectors that differ only by a nonzero complex scalar multiple

[1] This overview is based on the exposition in Nielsen and Chuang (2010, Chapter 2).

represent the same physical state. In particular, $|\psi\rangle$ and $e^{i\theta}|\psi\rangle$ are physically indistinguishable for any real θ.

Postulate 2 (Time evolution of closed systems). The time evolution of a closed system is described by a unitary operator on its state space. If the state at time t_1 is $|\psi(t_1)\rangle$ and the state at time t_2 is $|\psi(t_2)\rangle$, then there exists a unitary operator U on $\mathcal{H}$ such that

$$|\psi(t_2)\rangle = U|\psi(t_1)\rangle, \tag{1.1}$$

where $U^\dagger U = I$. Unitary evolution preserves inner products and, in particular, the normalization of state vectors.

Postulate 3 (Measurement). Let $\{|i\rangle\}$ be an orthonormal basis of the state space $\mathcal{H}$, and consider a measurement in this basis. Any state can be written as

$$|\psi\rangle = \sum_i \alpha_i |i\rangle, \qquad \sum_i |\alpha_i|^2 = 1. \tag{1.2}$$

The measurement has discrete outcomes labeled by i, and quantum mechanics assigns:

- probability
$$p(i) = |\alpha_i|^2 = |\langle i|\psi\rangle|^2 \tag{1.3}$$
to obtaining outcome i;

- post-measurement state
$$|\psi_i\rangle = |i\rangle \tag{1.4}$$
conditioned on observing outcome i.

Equivalently, this measurement can be described by the family of orthogonal projectors $P_i = |i\rangle\langle i|$, with $p(i) = \langle\psi|P_i|\psi\rangle$ and normalized post-measurement states $|\psi_i\rangle = P_i|\psi\rangle/\sqrt{p(i)}$. More general measurements can be modeled by a collection of measurement operators $\{M_m\}$ satisfying $\sum_m M_m^\dagger M_m = I$, but in this thesis we will primarily use projective measurements in the computational basis.

Postulate 4 (Composite systems). For a composite system consisting of subsystems A and B with state spaces $\mathcal{H}_A$ and $\mathcal{H}_B$, the state space of the combined system is the tensor product

$$\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B. \tag{1.5}$$

If $|\psi\rangle_A \in \mathcal{H}_A$ and $|\phi\rangle_B \in \mathcal{H}_B$ are states of the individual subsystems, then the product state of the composite system is $|\psi\rangle_A \otimes |\phi\rangle_B$. More generally, a pure state of the composite system is any unit vector $|\Psi\rangle_{AB} \in \mathcal{H}_A \otimes \mathcal{H}_B$, which can be expressed as

$$|\Psi\rangle_{AB} = \sum_{ij} \alpha_{ij} |i\rangle_A \otimes |j\rangle_B \tag{1.6}$$

with respect to chosen orthonormal bases $\{|i\rangle_A\}$ and $\{|j\rangle_B\}$. A pure state is called *separable* if it can be written as $|\Psi\rangle_{AB} = |\psi\rangle_A \otimes |\phi\rangle_B$ for some $|\psi\rangle_A$ and $|\phi\rangle_B$, otherwise *entangled*. Even when entangled qubits are spatially separated, a measurement on one qubit instantaneously fixes the conditional state of the others: once an outcome is obtained on one side, the outcome probabilities on the other side are updated accordingly. For example, if

$$|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix}, \quad |\phi\rangle = \begin{pmatrix} c \\ d \end{pmatrix},$$

then their tensor product is:

$$|\psi\rangle \otimes |\phi\rangle = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}.$$

1.1.4 Quantum Computing

Classical computers represent information using *bits*, which can take values of either 0 or 1. Quantum computing generalizes this idea using *quantum bits*, or *qubits*.

A single qubit is a two-dimensional quantum system with state space $\mathcal{H} \cong \mathbb{C}^2$. Fixing the computational basis $\{|0\rangle, |1\rangle\}$, any pure state of a qubit can be written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1. \tag{1.7}$$

As global phases are irrelevant, because $|\psi\rangle$ and $e^{i\theta}|\psi\rangle$ describe the same physical state, we can use this to parametrize any qubit state by two real angles $\theta \in [0, \pi]$ and $\varphi \in [0, 2\pi)$ as

$$|\psi(\theta, \varphi)\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\theta}{2}\right)|1\rangle. \tag{1.8}$$

The pair (θ, φ) can be interpreted as spherical coordinates of a point on the unit sphere in $\mathbb{R}^3$. This represents the *Bloch sphere*: every qubit state

corresponds to a point on the surface of the sphere, with $|0\rangle$ and $|1\rangle$ at the north and south poles, respectively.

Unitary operators preserve inner products and norms, so they map pure states to pure states, corresponding to rotations.

A *single-qubit gate* is such a unitary operation applied to an individual qubit. Important examples include the Pauli operators

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \tag{1.9}$$

which generate π-rotations about the x-, y-, and z-axes of the Bloch sphere. The Pauli-X gate exchanges $|0\rangle$ and $|1\rangle$ and is the quantum analogue of a classical NOT gate. The Pauli-Z gate leaves $|0\rangle$ invariant and multiplies $|1\rangle$ by a phase -1, thereby changing relative phase without altering the measurement probabilities in the computational basis.

Another fundamental gate is the Hadamard operator

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \tag{1.10}$$

which maps basis states to equal superpositions,

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \tag{1.11}$$

The Hadamard gate is routinely used to create superposition states from computational basis states. More generally, one often considers continuous families of *rotation gates*

$$R_X(\theta) = e^{-i\theta X/2}, \quad R_Y(\theta) = e^{-i\theta Y/2}, \quad R_Z(\theta) = e^{-i\theta Z/2}, \tag{1.12}$$

which implement rotations of the Bloch vector by an angle θ around the corresponding axis, as well as phase gates such as

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \qquad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \tag{1.13}$$

which modify the relative phase between $|0\rangle$ and $|1\rangle$.

To describe multiple qubits within the formalism of Postulate 4, we use the tensor product to build composite systems. If each single qubit lives in a two-dimensional Hilbert space $\mathcal{H} \cong \mathbb{C}^2$ with basis $\{|0\rangle, |1\rangle\}$, then the state space of an n-qubit register is the tensor product

$$\mathcal{H}^{\otimes n} = \underbrace{\mathcal{H} \otimes \cdots \otimes \mathcal{H}}_{n \text{ times}} \cong \mathbb{C}^{2^n}. \tag{1.14}$$

The computational basis of $\mathcal{H}^{\otimes n}$ consists of all 2^n tensor products of single-qubit basis states,

$$\{|x_1\rangle \otimes \cdots \otimes |x_n\rangle : x_k \in \{0,1\}\}, \tag{1.15}$$

which are usually written more compactly as $|x_1 \cdots x_n\rangle$ for $x_1 \cdots x_n \in \{0,1\}^n$. Any pure n-qubit state can be expressed as a linear combination

$$|\Psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \qquad \sum_x |\alpha_x|^2 = 1. \tag{1.16}$$

A particularly simple class of multi-qubit states are *product states*, which factorize as a tensor product of single-qubit states, for example

$$|\Psi\rangle = |\psi_1\rangle \otimes \cdots \otimes |\psi_n\rangle, \tag{1.17}$$

with each $|\psi_k\rangle = \alpha_k|0\rangle + \beta_k|1\rangle$. However, Postulate 4 allows more general states that cannot be written in this factorized form, entangled states. Popular examples are the Bell states, such as

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\big(|00\rangle + |11\rangle\big). \tag{1.18}$$

Measuring the first qubit of $|\Phi^+\rangle$ in the computational basis yields outcome 0 or 1 with equal probability, but conditioned on the outcome, the state of the second qubit is perfectly correlated: if the first outcome is 0 the joint state collapses to $|00\rangle$, whereas if the first outcome is 1 it collapses to $|11\rangle$. These correlations cannot be explained by any classical product state and are stronger than those obtainable from independent systems with shared classical randomness.

Entanglement plays a central role in quantum computation because it allows a quantum computer to represent and manipulate correlations across many subsystems in a way that has no direct classical analogue. In an n-qubit register, generic pure states live in a space of dimension 2^n, and entangled states can encode joint amplitude patterns over all 2^n basis strings.

It is important to distinguish entanglement from superposition. Superposition already appears at the single-qubit level: a state of the form $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ is a superposition of basis states, but not entangled, because it concerns only one system. Every entangled state is also a superposition, but not every superposition is entangled.

A quantum circuit consists of a sequence of quantum gates acting on a register of qubits. The structure of a quantum circuit reflects the algorithmic logic of the computation, with qubits initialized in known basis states,

processed through a network of gates, and finally measured to extract classical information. Quantum algorithms exploit this structure by applying multi-qubit gates that create and transform entanglement, so that interference between many computational paths can be steered towards desired outcomes.

Grover's Algorithm

To illustrate the operation of a quantum circuit, consider Grover's algorithm (Grover, 1996), which provides a quadratic speedup for the problem of searching an unstructured database. Grover's algorithm works as followed: given a function $f : \{0,1\}^n \to \{0,1\}$ that evaluates to $f(x) = 1$ for a unique input $x = x_0$, and $f(x) = 0$ otherwise, the goal is to find x_0. Classically, this requires $\mathcal{O}(2^n)$ evaluations in the worst case, whereas Grover's algorithm achieves this in $\mathcal{O}(\sqrt{2^n})$ steps.

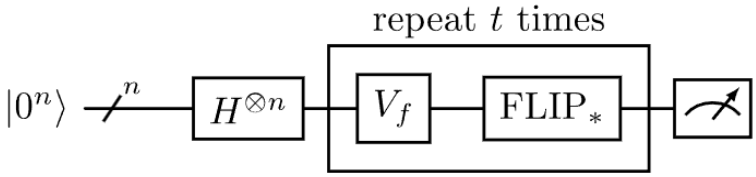

Figure 1.1: The quantum circuit for Grover's algorithm

1. **Initialization**

   We begin with an n-qubit register initialized to the state $|0\rangle^{\otimes n}$. Applying the Hadamard transform $H^{\otimes n}$ yields the uniform superposition:

   $$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \{0,1\}^n} |x\rangle, \quad \text{where } N = 2^n.$$

   This state assigns equal probability amplitude to every possible input $x \in \{0,1\}^n$.

2. **Oracle Operator**

   The oracle V_f is a unitary operator that flips the sign of the amplitude of the solution x_0. Formally:

   $$V_f|x\rangle = \begin{cases} -|x\rangle & \text{if } x = x_0, \\ |x\rangle & \text{otherwise.} \end{cases}$$

This phase inversion does not reveal x_0, but encodes its identity into the quantum state.

3. **Diffusion Operator or FLIP$_*$**

   The diffusion operator, often referred to as the inversion about the average, amplifies the probability amplitude of the solution. It is defined as:

$$D = 2|\psi\rangle\langle\psi| - I,$$

   where $|\psi\rangle$ is the initial uniform superposition and I is the identity matrix. When applied to a state $|\phi\rangle$, the operator reflects $|\phi\rangle$ about $|\psi\rangle$.

4. **Grover Iteration**

   One Grover iteration consists of applying the oracle followed by the diffusion operator:

$$G = D \cdot V_f.$$

   The state after t iterations is:

$$|\psi^{(t)}\rangle = G^t|\psi\rangle.$$

   It can be shown that each application of G rotates the state vector closer toward the solution state $|x_0\rangle$ in a two-dimensional subspace spanned by $|x_0\rangle$ and its orthogonal complement. The rotation angle θ satisfies:

$$\cos(\theta) = \sqrt{\frac{N-1}{N}}, \quad \text{so } \theta \approx \frac{1}{\sqrt{N}} \text{ for large } N.$$

   To maximize the probability of measuring x_0, the number of iterations should be approximately:

$$t = \left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor.$$

5. **Measurement and Success Probability**

   After applying t iterations, we measure the quantum state in the computational basis. With high probability (approaching 1 as $N \to \infty$),

the outcome will be the desired value x_0. The overall complexity of the algorithm is therefore $\mathcal{O}(\sqrt{N})$, providing a quadratic speedup over classical brute-force search.

1.2 Classical Machine Learning

Supervised learning is a core approach in machine learning in which the goal is to infer a mapping from inputs to outputs using a dataset of labeled examples. Each example is represented as a pair $(\mathbf{x}, y)$, where $\mathbf{x} \in \mathbb{R}^n$ is an input vector composed of n features, and y is the corresponding target value. The learning algorithm or mathematical function, the model, attempts to approximate a function $f : \mathbb{R}^n \to \mathcal{Y}$ that best predicts y given $\mathbf{x}$, where $\mathcal{Y}$ denotes the set of all possible target values, also called the output or label space (Murphy, 2012).

Some models in supervised learning are called *parametric*, meaning they assume a fixed form for the function f and learn a set of parameters, typically denoted by a weight vector $\mathbf{w} \in \mathbb{R}^n$ and a bias term $b \in \mathbb{R}$ during training. Others are *non-parametric*, which means their structure or complexity grows with the amount of data, without assuming a fixed number of parameters (Murphy, 2012).

In classification tasks, models often produce an intermediate output $z = \mathbf{w}^\top \mathbf{x} + b$, called the score. This is a real-valued scalar that serves as a bridge between raw input features through a decision or *activation function* to final output labels $\hat{y} \in \mathcal{Y}$. For example, a *sign function* $\hat{y} = \text{sign}(z)$ returns $+1$ or -1 based on the sign of z, while probabilistic models may use sigmoid or softmax functions to produce class probabilities. The geometry and dimensionality of the feature space (i.e., how many and which features are used to describe each $\mathbf{x}$) deeply influence how models learn and generalize.

1.2.1 Linear Classifiers

Linear classifiers are a class of models that separate data points in a feature space using a hyperplane defined by a weight vector $\mathbf{w} \in \mathbb{R}^n$ and a scalar bias term $b \in \mathbb{R}$. The model computes a linear combination $z = \mathbf{w}^\top \mathbf{x} + b$, and the sign of z determines the predicted class. This leads to a decision function of the form $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$, where $\text{sign}(\cdot)$ returns $+1$ for positive inputs and -1 for negative ones (Rosenblatt, 1958).

The vector $\mathbf{w}$ encodes the relative importance of each feature in the input $\mathbf{x}$, while the bias b shifts the decision boundary. The decision boundary itself is the set of points $\mathbf{x}$ for which $\mathbf{w}^\top \mathbf{x} + b = 0$. Linear classifiers rely on

the assumption that classes can be separated by such a boundary in the input space (Rosenblatt, 1958). They form the basis of many more complex models and are computationally simple, making them a foundational concept in machine learning.

1.2.2 Support Vector Machine (SVM)

While linear classifiers separate the classes by finding any line, which makes them sensitive to outliers, support vector machines (SVMs) (Cortes and Vapnik, 1995), by construction, are margin-based classifiers that aim to find the hyperplane separating two classes, and not focus on outlier points. This makes them more outlier-robust. Given training examples labeled $y_i \in \{-1, +1\}$, the SVM solves an optimization problem that finds the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ such that the distance between this hyperplane and the closest data points (called *support vectors*) is maximized.

Maximizing the margin improves the model's robustness to variations in the data. The supporting hyperplanes, those defining the margin boundaries, are given by $w^T x + b = \pm 1$. The distance from the decision boundary to each of these hyperplanes is $\frac{1}{\|w\|}$. Therefore, the total margin width, which is the distance between the two margin hyperplanes, is $\frac{2}{\|w\|}$. Maximizing it is equivalent to minimizing $\|\mathbf{w}\|^2$, which leads to the primal optimization formulation. This choice regularizes the model by penalizing large weights, which in turn reduces sensitivity to small fluctuations in input values.

The model predicts labels using the *sign function*: $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. In this setting, the sign of the inner product determines on which side of the hyperplane the input lies. If the data are not linearly separable, SVMs can be extended using kernel functions, which allow the model to behave as if the data were mapped into a higher-dimensional space, where a linear separator may exist, without explicitly computing that mapping.

1.2.3 Kernel Methods

Kernel methods (Schölkopf and Smola, 2002) are a set of techniques that allow linear models to learn non-linear patterns by computing similarities between data points using a *kernel function*. A kernel is a function $K(\mathbf{x}, \mathbf{x}')$ that measures how similar two inputs are, without explicitly mapping them into a higher-dimensional space. Instead, it leverages the mathematical property that inner products in this high-dimensional space can be computed directly via the kernel, a concept known as the *kernel trick*.

One widely used kernel is the Gaussian kernel or RBF(Radial Basis Function):

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

This function assigns high similarity to points that are close in Euclidean space and low similarity to distant points. This similarity measure enables the model to construct complex, non-linear decision boundaries in the input space, while still solving a linear problem in an implicit feature space.

1.2.4 Neural Networks (MLPs)

Multilayer Perceptrons (MLPs) (Goodfellow et al., 2016) are a class of feed-forward neural networks consisting of multiple layers of *artificial neurons*. Each neuron computes a weighted sum of its inputs, applies a non-linear *activation function* such as the rectified linear unit (ReLU), sigmoid, or hyperbolic tangent (tanh), and passes the result to the next layer. An MLP typically consists of an *input layer*, one or more *hidden layers*, and an *output layer*.

Formally, each hidden layer performs a transformation of the form:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases of layer l, and $\sigma(\cdot)$ is the activation function. The model is trained using *backpropagation*, which computes gradients of a loss function with respect to each parameter using the chain rule, followed by *gradient descent* to update the weights.

Small MLPs, typically with one or two hidden layers and a moderate number of units, are capable of learning complex, non-linear relationships while maintaining tractability. Unlike kernel methods or instance-based models, MLPs *learn internal representations* of the data during training, which makes them a flexible and general-purpose modeling approach.

Convolutional Neural Networks (CNNs). For image-like data, a common variant of neural networks is the convolutional neural network (CNN). Instead of fully connecting every input pixel to every neuron in the first layer, CNNs use *convolutional layers*, which apply small learnable filters locally across the input. The same filter is reused at all spatial positions, so the network can detect the same pattern (edges, lines or corners) regardless of where it appears in the image. Convolutional layers are typically followed by non-linear activation functions and *pooling* operations that downsample the feature maps, gradually reducing spatial resolution while increasing the level

of abstraction. In the final stages, one or more fully connected layers map these extracted features to class scores, making CNNs particularly effective for image classification tasks.

1.3 Quantum Machine Learning

1.3.1 How data is encoded?

Classical data can be embedded into quantum states in several ways; here we briefly summarise a few common schemes following Rath and Date (2023).

Basis (computational) encoding. The most direct method maps each classical bit to a qubit in the computational basis. A classical bit string such as 101 is represented as the basis state

$$|101\rangle = |1\rangle \otimes |0\rangle \otimes |1\rangle.$$

If we measure in the computational basis, we always obtain the string 101; the information is stored in *which* basis state the system occupies.

Superposition encoding. Instead of a single basis state, one can encode information in a superposition of several classical strings. For example,

$$|\psi\rangle = \frac{1}{\sqrt{3}}\big(|100\rangle + |010\rangle + |001\rangle\big)$$

encodes that the system is in one of the three strings 100, 010, or 001 with equal probability. Measurement yields one of these outcomes with probability 1/3, so the information is contained in which basis states appear and with which amplitudes.

Angle encoding. In angle (or rotation) encoding, real-valued features are used as rotation angles on single qubits. For a two-dimensional data point $x = (x_1, x_2)$, one can prepare

$$|\psi(x)\rangle = R_y(x_1) \otimes R_y(x_2)\,|00\rangle = R_y(x_1)|0\rangle \otimes R_y(x_2)|0\rangle,$$

where $R_y(x)$ is a rotation about the y-axis by angle x. The measurement probabilities then depend smoothly on the input values x_1 and x_2. Analogy: each qubit is an arrow on the Bloch sphere. Your data tells you how much to turn the arrow.

Amplitude encoding. Amplitude encoding uses the components of a (normalised) classical vector as the amplitudes of a quantum state. For instance, for $x = (1, 2, 0, 1)$ we first normalise $\|x\| = \sqrt{1^2 + 2^2 + 0^2 + 1^2} = \sqrt{6}$ and prepare the two-qubit state

$$|\psi(x)\rangle = \frac{1}{\sqrt{6}}|00\rangle + \frac{2}{\sqrt{6}}|01\rangle + 0 \cdot |10\rangle + \frac{1}{\sqrt{6}}|11\rangle.$$

Here the classical information is encoded directly in the probability amplitudes of each basis state. The classical numbers (1,2,0,1) decide how likely each basis state is, after the normalisation.

1.3.2 Ansatz: How is the model learning?

After encoding, the quantum state is processed by a variational ansatz, i.e. a quantum circuit $U_{\text{var}}(\theta)$ with a fixed gate layout and fixed parameters or trainable angles θ. The ansatz specifies how many layers of single-qubit rotations and entangling gates are used, on which qubits they act, and how the qubits are connected. In this sense, it plays a role analogous to the architecture of a classical neural network: choosing the number of layers, the width of each layer, and the connectivity between neurons. While the feature map determines how the classical input x is embedded into the Hilbert space, the ansatz provides the trainable degrees of freedom that are optimized during learning and thus defines the family of functions that the model can represent (Benedetti et al., 2019).

1.3.3 Models in Quantum Machine Learning

Following the models introduced in (Bowles et al., 2024), we now discuss the following:

VQC's: Variational Quantum Circuits

In the setting of quantum machine learning, variational quantum circuits or VQC's are often described abstractly as functions of the form

$$f(\theta, x) = \operatorname{tr}\big[O(x, \theta)\, \rho(x, \theta)\big],$$

where $\rho(x, \theta)$ is a density matrix and $O(x, \theta)$ is an observable. This expression can be understood operationally as a three–step procedure. First, a feature map (encoding circuit) $U_{\text{enc}}(x)$ prepares a data-dependent quantum state from a simple initial state ρ_0, for example $|0 \ldots 0\rangle\langle 0 \ldots 0|$. This yields the

encoded state $U_{\text{enc}}(x)\,\rho_0\,U_{\text{enc}}(x)^\dagger$. Second, a variational ansatz $U_{\text{var}}(\theta)$, i.e. a parametrized quantum circuit with fixed gate layout and trainable angles θ, is applied to this encoded state. The result is the joint data- and parameter-dependent state, or the density matrix,

$$\rho(x,\theta) = U_{\text{var}}(\theta)\,U_{\text{enc}}(x)\,\rho_0\,U_{\text{enc}}(x)^\dagger U_{\text{var}}(\theta)^\dagger.$$

Finally, one measures a suitable observable $O(x,\theta)$ on this state. The scalar output is given by the trace or $f(\theta,x)$=tr$[O(x,\theta)\rho(x,\theta)]$, which is the expectation value of the measurement and serves as the prediction of the quantum model for input x and parameters θ.

Quantum Neural Networks

We view a quantum neural network as in (Bowles et al., 2024) as a model that uses one or more variational quantum circuits $f_1,\ldots,f_L$ and combines their scalar outputs through a classical prediction function f_{pred} to produce a binary label $y \in \{\pm 1\}$:

$$y = f_{\text{pred}}\big(f_1(\theta_1,x),\ldots,f_L(\theta_L,x)\big).$$

In the simplest case, we use only one quantum circuit ($L=1$) and measure an operator whose possible outcomes are -1 or $+1$. The circuit output is then a number between -1 and $+1$ (its expectation value), and we choose the predicted class just by its sign: a positive value means class $+1$, while a negative value means class -1.

Training proceeds analogously to classical neural networks. Given a dataset $\{(x_i,y_i)\}_{i=1}^N$, we define an average loss as

$$\mathcal{L}(\theta;X,y) = \frac{1}{N}\sum_{i=1}^{N}\ell\big(\theta,x_i,y_i\big),$$

where $\ell(\theta,x_i,y_i)$ is a pointwise loss and (X,y) collect all inputs and labels. The parameters $\theta=(\theta_1,\ldots,\theta_L)$ are then optimised using stochastic gradient descent or related methods. Gradients with respect to quantum circuit parameters are obtained via the chain rule and parameter-shift rules (Mitarai et al., 2018; Schuld et al., 2019).

Quantum Kernel Methods

Quantum kernel methods adapt classical kernel-based learning (such as support vector machines) to the quantum setting by using a quantum device

to define and evaluate the kernel function. The key idea is to embed each classical data point x into a quantum state $\rho(x)$ via a feature map (encoding circuit), and to define the kernel as an inner product between such states, for example

$$k(x_i, x_j) = \mathrm{tr}\left[\rho(x_i)\,\rho(x_j)\right].$$

This quantum kernel can then be plugged into a standard classical kernel algorithm, which operates exactly as in the classical case but now uses similarities computed by a quantum circuit. In this way, the "model" remains a classical kernel method, while the quantum computer is used only to realise a potentially rich feature space that may be hard to simulate classically.

1.4 Motivation: The Benchmarking Problem in QML

1.4.1 The benchmarking problem in quantum machine learning

Before large, fault-tolerant quantum computers are available, almost all empirical evidence about quantum machine learning (QML) comes from classical simulations of small quantum models or from experiments on noisy, few-qubit devices. In this regime, *benchmarking*, systematically comparing quantum models against each other and against classical baselines on common tasks, is one of the few tools available to assess whether current ideas in QML actually deliver on their promised advantages.[2]

However, as Bowles et al. (2024) argue, drawing robust conclusions from such benchmarks is surprisingly difficult: small design choices in datasets, model architectures, metrics, and hyperparameter tuning can drastically change which model "wins". This makes it hard to answer even seemingly simple questions such as whether a given quantum model is "better than" a classical one.

- **Dataset choice has a huge impact.** Classical "no free lunch" results already tell us that average performance of any learning algorithm depends strongly on the distribution of tasks considered. In practice, this means that simply changing which datasets are included in a benchmark, or how they are preprocessed, can completely reorder the ranking

[2]See Bowles et al. (2024) for a detailed discussion and extensive references to the benchmarking literature both in classical and quantum machine learning.

of methods. Empirically, even for standard classical models, fixing label errors in a dataset, changing the train–test split, or switching to a different evaluation metric can significantly shift which model appears best. In the QML setting, where many studies rely on only one or two small datasets, this sensitivity is even more problematic. Papers that illustrate this effect include (Dehghani et al., 2021; Northcutt et al., 2021; Recht et al., 2018, 2019).

- **Benchmark design embeds many hidden choices.** Beyond the data itself, numerous "small" design decisions shape the outcome:
  - Architectural tweaks (new ansätze, exotic layers, etc.) are often advertised as innovations, but large-scale benchmarks suggest that they frequently have little consistent effect on performance.
  - Hyperparameter optimisation (learning rates, layer counts, regularisation, kernel parameters, etc.) typically dominates performance: with enough tuning, a simple baseline can match or surpass more elaborate proposals.
  - Different evaluation metrics (accuracy, F1-score, AUC, calibration measures, or cost-sensitive scores) reward different behaviour and may correlate poorly, so "best" can mean different things depending on what is measured.
  - When computational cost is taken into account (e.g., training time, circuit evaluations, required hardware or simulator resources), rankings can reverse: a slightly more accurate model may be clearly inferior when efficiency is factored in.

  For QML, these issues are amplified because each circuit evaluation is expensive and gradients can require thousands of shots or full-state simulations, so hyperparameter searches are hardware- and time-intensive. Papers that illustrate this effect include (Narang et al., 2021; Smith et al., 2023; Lucic et al., 2018; Henderson et al., 2017; Riquelme et al., 2018; Dodge et al., 2019).

- **Systematic positivity bias in the literature.** A simple thought experiment illustrates a serious publication bias: suppose many research groups each try a large number of quantum models and datasets, but only submit papers when they find a configuration where the quantum model outperforms a classical baseline. Even if, on average, classical models tend to do better, the literature will still be dominated by positive results that claim "quantum beats classical". This creates a mis-

leading impression of steady progress, even if the underlying picture is much more mixed.

The conclusion is that broad questions of the form "Are quantum models better than classical ones?" cannot be answered by isolated experiments on a handful of small datasets. Instead, what can be meaningfully asked are *narrow*, well-specified questions: for example, how the performance of a particular family of quantum models changes as a controlled difficulty parameter increases, or how sensitive a given ansatz is to noise, or how a quantum kernel compares to a specific classical kernel on a particular class of tasks.

Carefully designed benchmarks must therefore emphasise controlled variation, reproducibility, and ablation-style comparisons over simple leaderboard rankings.

What matters in a comparison? Given these challenges, Bowles et al. (2024) argue that benchmarking should move beyond simplistic leaderboard questions and instead ask *structural* questions about model design. Rather than only asking whether a quantum model outperforms a classical baseline, we should try to understand *which parts* of the model are crucial and which can be replaced by classically simulable components without loss of performance. In other words, benchmarking should be used as an *ablation tool* to probe the role of quantum resources.

Concretely, they propose constructing "dequantised" variants of QML models by systematically removing or restricting those ingredients that are specifically quantum, for example by

- removing entangling gates and using only local, single-qubit rotations (seen in the paper),
- restricting the gate set to Clifford transformations,
- replacing unitary evolutions by stochastic, or
- simulating circuits with tensor-network methods such as Matrix Product States (MPS) with low bond dimension.

By comparing the original model to these non-quantum or classically tractable variants on the *same* benchmarks, one can probe to what extent genuine quantum effects are responsible for any observed performance differences. When a dequantised variant matches the original model, this suggests that "quantumness" was not the decisive ingredient, at least on the tasks and scales considered.

Motivated by this perspective, the present work adopts a similar philosophy. Rather than taking the presence of quantum circuits as evidence of a quantum advantage, we use benchmarking as a tool to dissect our models. After introducing baseline variational quantum classifiers, we will *remove* its specifically quantum features and study how performance changes. In particular, we investigate variants *without entanglement* and with other classically simulable restrictions, following the dequantisation strategies suggested by Bowles et al. (2024). By comparing the original and dequantised models on controlled benchmarks, our goal is to understand not only *how well* they perform, but also *which aspects of their design truly require quantum resources*.

Chapter 2

Methodology

This chapter describes the experimental setup used to study the role of entanglement in two quantum model families: the Circuit-Centric Classifier (CCC) and the IQP kernel classifier. The main goal is not only to compare models by final accuracy, but to construct controlled ablations of each architecture and examine how performance changes as entanglement is reduced. To keep the comparison as fair as possible, all variants are evaluated within the same benchmarking framework, on the same datasets, and under the same training and hyperparameter-search procedure.

2.1 Experimental Setup

All experiments are carried out within the benchmarking framework of Bowles et al. (2024), which provides a common interface for datasets, model evaluation, hyperparameter search, etc. In this thesis, I keep the overall benchmark structure of the paper and study two model families implemented in the same codebase: the Circuit-Centric Classifier introduced by Schuld et al. (2018) and the IQP kernel classifier. For both families, the goal is to compare the original model against entanglement-reduced variants under matched benchmark conditions.

The practical motivation for this setup is straightforward: if the original model and the modified variants are trained under identical conditions, then differences in performance can be attributed more directly to architectural changes, rather than to differences in preprocessing, tuning budget, or evaluation protocol. This is especially important in QML benchmarks, where small implementation choices can otherwise dominate the comparison (Bowles et al., 2024).

2.2 Datasets

This thesis uses the same benchmark family as the reference study of Bowles et al. (2024). The intention is therefore not to introduce new datasets, but to reuse the same controlled tasks so that the effect of dequantization can be studied in a setting that is already established in the literature.

The benchmark suite contains several dataset families, each designed to probe a different aspect of model behavior. Rather than relying on a single task, the benchmark varies either the input dimension, the intrinsic structure of the data, or the geometric complexity of the decision boundary. This makes it possible to ask more specific questions than whether a model performs well on average.

At one end of the spectrum, the **linearly separable** benchmark serves as the simplest sanity-check task in the suite. The two classes are separated by a single hyperplane, so a basic linear classifier should already solve the problem. Its main role is therefore to test whether a model can recover a simple linear decision rule as the input dimension changes.

The **bars and stripes** benchmark moves from geometric toy data to simple image structure. Samples are noisy images belonging either to the class of vertical bars or to the class of horizontal stripes. As this benchmark requires exponentially more ressource to test, we prescind from it for our experiment.

The **downscaled MNIST** benchmark provides a more realistic classification task while remaining tractable for QML simulations. It uses the handwritten-digit problem restricted to digits 3 versus 5, but in reduced form. In the reference benchmark, this appears both as low-dimensional PCA representations and as coarse-grained or lower resolution image representations for convolutional models. This dataset family is useful because it bridges the gap between highly controlled synthetic tasks and simplified real-image data.

The **hidden manifold** benchmarks are designed to mimic settings in which the observed data live or is in a high-dimensional space even though the true structure, or the manifold, is governed by fewer latent degrees of freedom, as is often the case in image and signal data. Two variants are particularly relevant. In **hidden manifold model**, the manifold dimension is held fixed while the visible input dimension increases. The underlying problem therefore remains the same, but it is embedded into a larger ambient space. In **hidden manifold diff**, by contrast, the visible dimension is fixed while the manifold dimension changes. Here the observed number of features stays constant, but the latent structure itself becomes more complex, making the task intrinsically harder.

The **two curves** benchmark tests nonlinear geometric classification by

generating each class from a separate one-dimensional curve. These curves are embedded into a d-dimensional feature space. The curves are constructed using low-degree Fourier series, meaning combining sine and cosine functions, which makes smooth nonlinear structures, and Gaussian noise is added so points lie near the curves. The task is to classify a new point according to which of the two curves it most likely originated from. Two curves fixes the curve complexity and the offset between the curves, but increases the feature dimension d. Two curves diff fixes the feature dimension at d=10 but makes the curves more complex and closer together.

The **hyperplanes and parity** benchmark defines labels by combining several hyperplane tests. For each sample, the model must determine on which side of each hyperplane the point lies, and the final class is given by whether the number of positive-side outcomes is even or odd. This makes the task compositional, because the label depends on the combination of several simple boundary relations rather than on a single separating hyperplane. In the hyperplanes diff variant, the task becomes harder by increasing the number of hyperplanes that must be combined.

Taken together, these dataset families provide a controlled progression from simple linear tasks to problems with hidden low-dimensional structure, curved decision geometry, and compositional parity rules. This is important for the present thesis because it allows the role of entanglement to be studied across qualitatively different learning scenarios, rather than being inferred from a single benchmark alone.

2.3 Circuit-Centric Classifier

The first quantum model family studied in this thesis is the Circuit-Centric Classifier (CCC), following the architecture proposed by Schuld et al. (2018). It follows the general ideas of data embedding, variational ansätze, and measurement-based predictions, which were introduced in the previous chapter.

2.3.1 Input Encoding

CCC uses amplitude encoding together with repeated copies of the encoded input state. In practice, if the classical feature dimension is not already a power of two, the input is padded accordingly. Then, the resulting state is embedded multiple times through a tensor-product construction. If d denotes the number of classical input features and C denotes the number of input

copies, then each amplitude-encoded register requires

$$\lceil \log_2(d) \rceil$$

qubits. Therefore, the total number of qubits in the model is

$$M = C \cdot \lceil \log_2(d) \rceil .$$

.

This part of the model is inherited directly from the original CCC and is identical for the original and dequantized variants.

2.3.2 Variational Circuit and Prediction Rule

This is the part of the tensor product construction. In the reference implementation used here, the original CCC employs PennyLane's `StronglyEntanglingLayers` as its variational block. The model output is computed from the expectation value of the Pauli-Z observable measured on the first qubit. This expectation value is a scalar in the interval $[-1, 1]$, to which a trainable bias term b is added to obtain the final score. For the purposes of this thesis, the important point is that the prediction rule is kept fixed across all CCC variants. The only systematic modification concerns the internal entangling structure of the variational circuit.

2.4 Dequantized CCC Variants

The methodological contribution of this thesis is not a new model family, but a controlled modification of CCC designed to reduce or remove entanglement while preserving the rest of the architecture as far as possible. This follows the ablation perspective advocated by Bowles et al. (2024).

To this end, I introduce dequantized CCC variants that keep the same embedding, the same measurement rule, the same training pipeline, and the same overall role of the variational block, but replace the original ansatz by a custom controlled construction in which the amount of entanglement can be adjusted directly.

2.4.1 Controlled Ansatz

The modified ansatz is built layer by layer. Each layer first applies a trainable general single-qubit rotation on every qubit,

$$\mathrm{Rot}(\alpha, \beta, \gamma),$$

so the local trainable degrees of freedom are retained in all variants. The difference between variants lies only in how many of the candidate two-qubit couplings are actually activated.

Let M be the total number of qubits. In layer ℓ, the candidate entangling edges are defined by a cyclic offset

$$r_\ell = (\ell \bmod (M-1)) + 1,$$

which produces the ring-like edge set

$$E_\ell = \{(i, i + r_\ell \bmod M) : i = 0, \ldots, M-1\}.$$

This means that successive layers cycle through different interaction ranges. From the candidate edge set, only a fraction is activated. If $\alpha \in [0, 1]$ denotes the entanglement-control parameter, then the number of active entangling gates in layer ℓ is

$$k_\ell = \text{round}(\alpha |E_\ell|).$$

Here, round$(\cdot)$ denotes rounding to the nearest integer. The active edges are chosen by seeded pseudorandom sampling, making the construction reproducible across runs while avoiding the same deterministic subset in every layer.

Operationally, α controls how much entangling structure is available to the circuit:

- $\alpha = 0$ removes all two-qubit gates and yields a fully separable circuit,
- $0 < \alpha < 1$ produces partially entangling circuits, and
- $\alpha = 1$ refers to all the entangling gates activated.

In the current implementation, the entangling gate can be chosen as either a controlled-NOT or a controlled-Z gate. This thesis does not compare gate primitives directly; the central methodological parameter is the fraction of active entangling edges.

2.4.2 Separable and Half-Separable Variants

Two dequantized variants are considered in this work.

The first is the *separable* CCC. Here the entanglement-control parameter is set to $\alpha = 0$. As a result, no two-qubit gates are applied anywhere in the variational block. The circuit therefore remains fully local, and this constitutes the strongest dequantization step considered in the thesis.

The second is the *half-separable* CCC. Here the entanglement-control parameter is set to $\alpha = 0.5$, so that approximately half of the candidate two-qubit couplings in each layer remain active. This model occupies an intermediate position between the original entangling CCC and the fully separable version. It is methodologically useful because it allows us to test whether performance degrades gradually as entanglement is reduced, or whether most of the observed behavior can already be reproduced with only limited entangling structure.

During development, a deterministic *first-50%* selection rule was also tested, in which the first half of the candidate couplings in each layer was kept active. However, this turned out to be a poor choice for the final comparison because it ties the definition of the model to the arbitrary ordering of edges in the circuit construction rather than only to the intended entanglement level. For this reason, the final half-separable variant uses a seeded random 50% subset instead, which preserves reproducibility while avoiding this ordering bias.

Taken together, the three models studied in this chapter form a simple entanglement ablation ladder:

- original CCC with a fully entangling benchmark ansatz,
- half-separable CCC with partial entangling connectivity, and
- separable CCC with no entangling gates.

2.5 IQP Kernel Classifier

The second quantum model family studied in this thesis is the IQP kernel classifier, based on the `IQPKernelClassifier` already present in the benchmark codebase. This family is included so that the entanglement-ablation question can be asked not only for a variational classifier, but also for a quantum-kernel model.

The IQP kernel model differs conceptually from CCC in an important way. CCC is a variational classifier with trainable circuit parameters and a measurement-based prediction rule. By contrast, the IQP-kernel model uses a fixed feature map to define a kernel

$$k(x, x') = |\langle 0|U(x')^\dagger U(x)|0\rangle|^2,$$

which is then passed to a classical support-vector machine with a precomputed kernel matrix. In other words, the quantum circuit defines the feature space, while the final classifier is a classical SVM.

In the implementation used here, the single-qubit part of the IQP feature map is kept fixed across all variants, and only the ZZ entangling pattern is changed. The hyperparameter interface is also kept aligned across variants, in particular the number of IQP repeats and the SVM regularisation parameter.

2.5.1 IQP Kernel Variants

Three IQP-kernel variants are considered.

The first is the original `IQPKernelClassifier`, which uses the standard IQP-style feature map with all ZZ couplings present in each repeat of the embedding.

The second is `IQPKernelClassifierSeparable`. This variant keeps the same single-qubit IQP encoding but removes all ZZ couplings. It is therefore the direct no-entanglement ablation for the IQP feature map.

The third is `IQPKernelClassifierHalfSeparable`. This variant again keeps the same single-qubit IQP encoding, but for each repeat it retains only a seeded random 50% subset of the full ZZ edge set. As in the CCC case, the intention is to test whether reducing entangler density causes a gradual loss relative to the full model, or whether much of the kernel's behavior can already be recovered with a sparse entangling pattern.

These IQP-kernel variants are methodologically useful because they allow the entanglement question to be posed in a different QML model class. In CCC, entanglement appears inside a trainable variational ansatz. In the IQP-kernel model, by contrast, the entangling ZZ structure is part of the fixed feature map itself. Comparing both families therefore helps separate claims about entanglement in a particular ansatz from claims about entanglement in a broader QML pipeline.

2.6 Training Procedure

All model variants are trained within the same benchmark pipeline. This includes the same dataset generation, the same train-test protocol, the same hyperparameter-search infrastructure, and the same result logging format. The purpose of this uniform setup is to ensure that the comparison between CCC and its dequantized variants is an ablation study of model structure rather than a comparison between different experimental conditions.

For CCC, the implementations are trained as binary classifiers with labels in $\{-1, +1\}$. The model parameters consist of the variational circuit parameters together with the scalar bias term. Hyperparameters such as the

number of input copies, the number of variational layers, optimization settings, and related training parameters are selected through the same search procedure used by the benchmark framework. This point matters because, as emphasized by Bowles et al. (2024), hyperparameter tuning can strongly affect benchmark outcomes in QML.

For the IQP-kernel family, the training flow is different but the benchmarking philosophy is the same. The quantum circuit is used to construct kernel matrices, and classification is then performed by a classical SVM backend. The main hyperparameters are the number of IQP repeats and the SVM regularisation parameter. The entanglement ablations are therefore applied at the level of the kernel feature map rather than at the level of a trainable variational circuit.

2.7 Evaluation

Model performance is evaluated primarily through classification accuracy on the benchmark tasks. Since the study is comparative rather than hardware-focused, the central object of interest is the relative ranking of the original and dequantized CCC variants across datasets and benchmark settings. The experiments therefore examine not only absolute scores, but also whether removing entanglement changes the ordering of models across tasks of different difficulty.

From the perspective of the thesis, the key interpretive question is the following: if a partially entangling or fully separable variant performs similarly to the original CCC, then the benefit of the original model cannot be attributed straightforwardly to entanglement. Conversely, if performance drops consistently as entanglement is restricted, this provides evidence that entangling structure is playing a nontrivial role in the model's inductive bias.

All reported results come from the benchmark framework described in this chapter. For each dataset instance, each model is trained with the same preprocessing, the same train-test split convention, and the same hyperparameter-search machinery. The primary metric used in the present analysis is test accuracy. Train accuracy is used only as a secondary diagnostic to assess whether changes in test performance are associated with changes in fitting capacity or with possible overfitting.

Chapter 3

Results

3.1 Quantitative Results

Across the completed dataset instances, the original entangling CCC is the strongest model overall. It achieves the best test accuracy on 90 instances, while the half-separable random-50 variant is best on 31 instances and the fully separable model is best on 13 instances. This already shows a clear hierarchy: the original model is usually strongest, the half-separable variant is often competitive, and the fully separable model is only occasionally best.

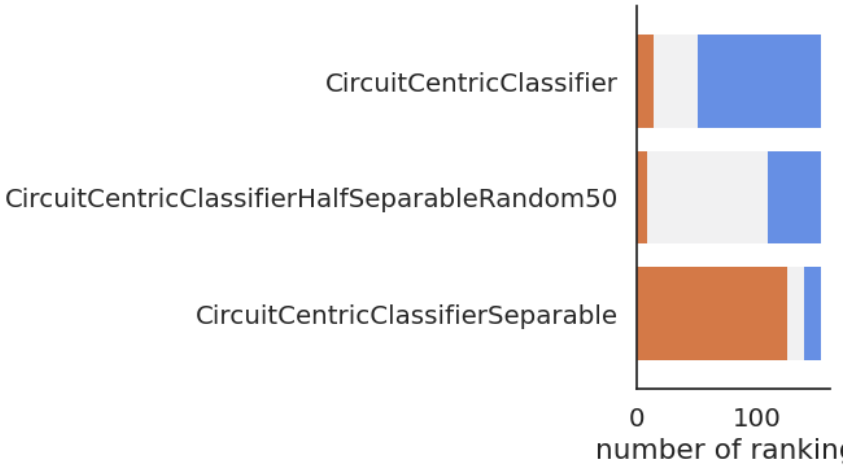

Figure 3.1: Aggregate ranking of the three CCC variants across the completed benchmark set.

A more informative picture comes from the average accuracy differences. Relative to the original CCC, the half-separable random-50 variant shows

only a small mean loss in test accuracy across the completed benchmark set, about -0.0127. By contrast, the fully separable model is markedly worse, with a mean test-accuracy difference of about -0.1638 relative to the original CCC. The difference between the half-separable and fully separable variants is correspondingly large, about $+0.1510$ in favor of the half-separable model. The main empirical pattern is therefore not that every reduction of entanglement is equally harmful, but that *removing entanglement entirely is much more damaging than thinning it out.*

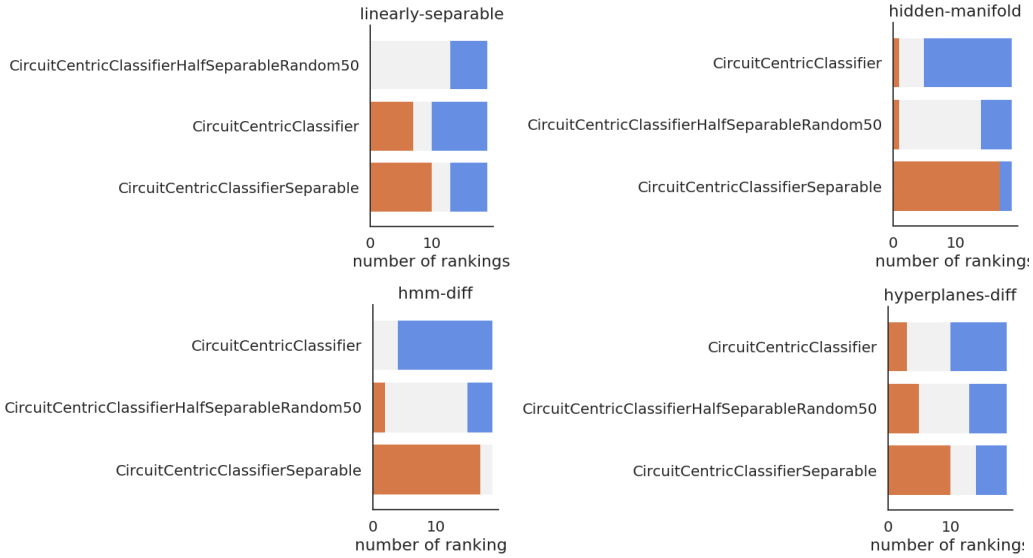

Figure 3.2: Per-family ranking plots for linearly separable, hidden manifold, hidden manifold diff, and hyperplanes diff.

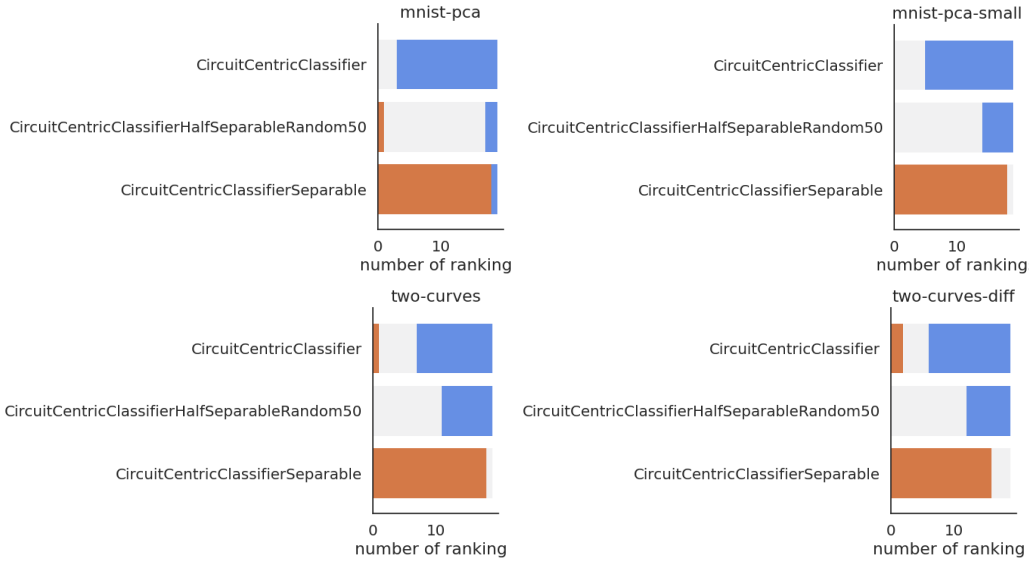

Figure 3.3: Per-family ranking plots for MNIST PCA, MNIST PCA small, two curves, and two curves diff.

This pattern is also visible when the results are broken down by benchmark family. On linearly separable data, the half-separable variant is slightly better on average than the original model, by about $+0.0091$, while the fully separable model is slightly worse, by about -0.0391. This suggests that for the simplest tasks the full entangling structure is not necessary and may even be mildly excessive. On hyperplanes diff, the three variants are comparatively close, with the separable model behind the original by only about -0.0598. By contrast, on hidden manifold, hidden manifold diff, MNIST PCA, MNIST PCA small, and two curves diff, the fully separable model is consistently and often substantially worse than the original CCC. The effect is especially pronounced on the MNIST PCA families, where the separable model trails the original by about -0.2934 and -0.2765, respectively.

The half-separable variant occupies an intermediate but important position. On most benchmark families it is slightly worse than the original CCC, but the drop is small compared with the fully separable case. For example, the average test-accuracy difference half-minus-full is about -0.0163 on hidden manifold, -0.0237 on hidden manifold diff, -0.0188 on hyperplanes diff, -0.0124 on MNIST PCA, -0.0088 on MNIST PCA small, and -0.0193 on two curves diff. These values are modest enough to suggest that a sparse entangling structure preserves a substantial fraction of the original model's performance.

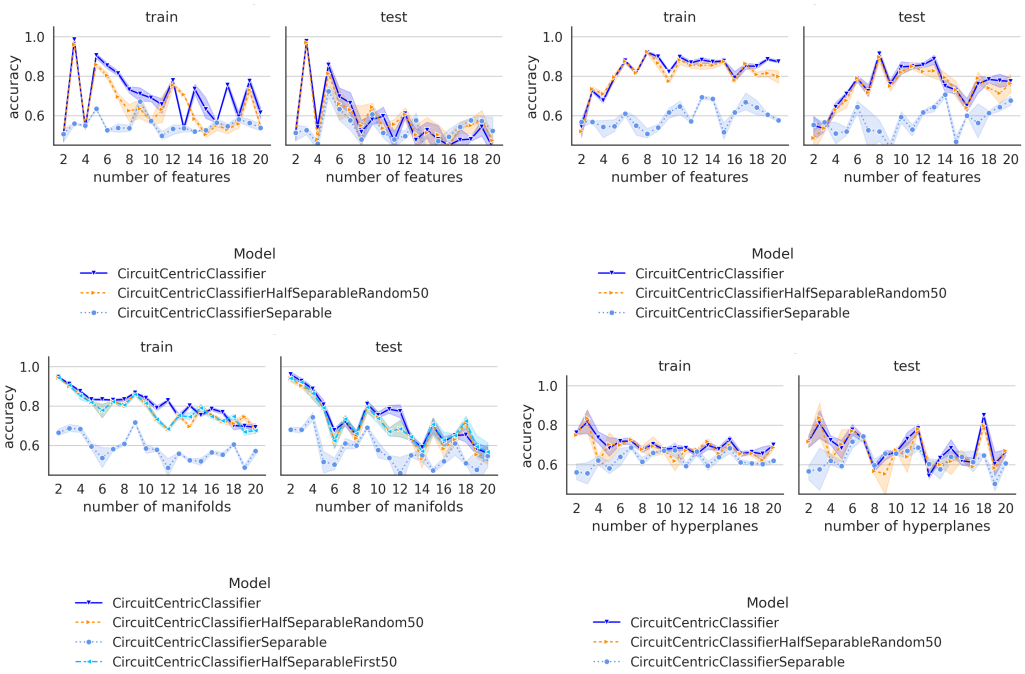

Figure 3.4: Train and test accuracy curves for the completed geometric benchmark families: linearly separable, hidden manifold, hidden manifold diff and hyperplanes diff.

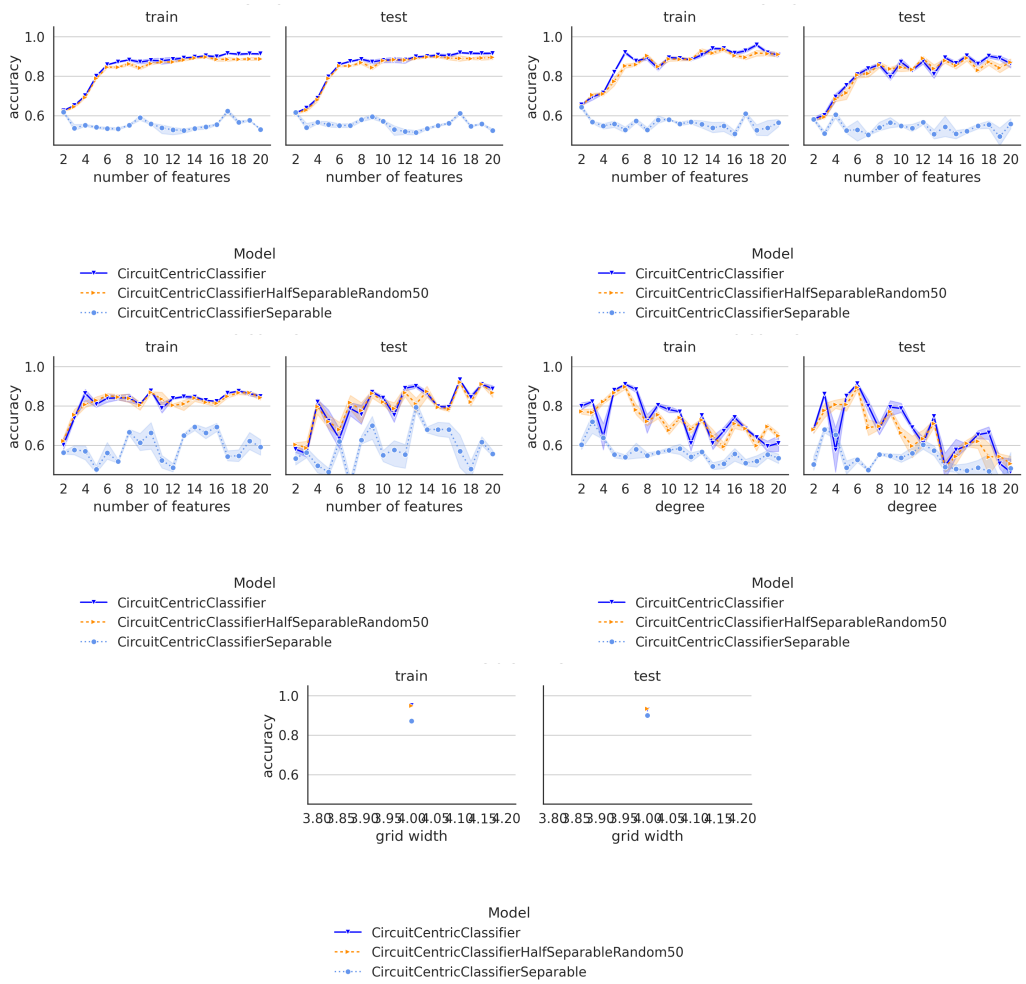

Figure 3.5: Train and test accuracy curves for MNIST PCA, MNIST PCA small, two curves, and two curves diff

The train-versus-test comparison is broadly consistent with this interpretation. Averaged over the completed benchmark set, the original CCC reaches about 0.7850 train accuracy and 0.7274 test accuracy, while the half-separable variant reaches about 0.7642 train accuracy and 0.7146 test accuracy. The separable model is much lower on both metrics, with about 0.5779 train accuracy and 0.5636 test accuracy. In other words, the separable model does not merely generalize worse after fitting the data equally well; it appears to have substantially lower effective fitting capacity on these tasks. This is especially visible on hidden manifold, MNIST, and two-curves benchmarks.

The main quantitative conclusion from the completed CCC data is therefore the following: the original entangling CCC is usually best, but the half-

separable random-50 variant remains surprisingly competitive, whereas the fully separable model often incurs a substantial loss. Entanglement appears to matter, but the completed results suggest that *full entangler density is often not required to obtain most of the performance of the original model.*

3.2 Qualitative Interpretation

Although the benchmark suite is synthetic and controlled, the dataset families probe meaningfully different kinds of structure, and the effect of entanglement is not uniform across them.

For linearly separable data, the results are unsurprising: a highly expressive entangling circuit is not needed to implement a linear decision rule. The fact that the half-separable model can slightly outperform the original CCC on this family is therefore plausible and should not be over-interpreted as evidence against entanglement. Instead, it suggests that the full model may be using more representational freedom than the task demands.

The hidden-manifold families tell a different story. There the fully separable model drops substantially, while the original and half-separable variants remain much closer. Since these benchmarks are intended to probe structure that is embedded nontrivially in a larger feature space, this pattern is consistent with the idea that some nonlocal interaction structure in the variational block helps the model align with the task geometry.

The same qualitative conclusion appears on the MNIST PCA families and on two curves diff. In these settings, eliminating entanglement entirely leads to a pronounced reduction in performance, while keeping a sparse random half of the entangling couplings preserves much more of the original score. In practical terms, this means that a moderate amount of entangling structure is enough to capture most of the gain, whereas a fully local circuit is often too weak.

Chapter 4

Conclusion and Outlook

Bibliography

Benedetti, M., Lloyd, E., Sack, S., and Fiorentini, M. (2019). Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001.

Bohr, N. (1913). On the constitution of atoms and molecules. *Philosophical Magazine*, 26:1–25.

Born, M. (1926). Zur quantenmechanik der stoßvorgänge. *Zeitschrift für Physik*, 37:863–867.

Bowles, J., Ahmed, S., and Schuld, M. (2024). Better than classical? The subtle art of benchmarking quantum machine learning models. *arXiv preprint arXiv:2403.07059*.

Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.

de Broglie, L. (1924). *Recherches sur la théorie des quanta*. Phd thesis, Université de Paris.

Dehghani, M., Tay, Y., Gritsenko, A. A., Zhao, Z., Houlsby, N., Diaz, F., Metzler, D., and Vinyals, O. (2021). The benchmark lottery. *arXiv preprint arXiv:2107.07002*. arXiv preprint arXiv:2107.07002.

Dodge, J., Gururangan, S., Card, D., Schwartz, R., and Smith, N. A. (2019). Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.

Einstein, A. (1905). Über einen die erzeugung und verwandlung des lichtes betreffenden heuristischen gesichtspunkt. *Annalen der Physik*, 17:132–148.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing (STOC 1996)*, pages 212–219. ACM.

Heisenberg, W. (1925). Über quantentheoretische umdeutung kinematischer und mechanischer beziehungen. *Zeitschrift für Physik*, 33:879–893.

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. (2017). Deep reinforcement learning that matters. *CoRR*, abs/1709.06560. arXiv preprint arXiv:1709.06560.

Lucic, M., Kurach, K., Michalski, M., Gelly, S., and Bousquet, O. (2018). Are GANs created equal? a large-scale study. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*.

Mitarai, K., Negoro, M., Kitagawa, M., and Fujii, K. (2018). Quantum circuit learning. *Physical Review A*, 98(3):032309.

Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press, Cambridge, MA.

Narang, S., Chung, H. W., Tay, Y., Fedus, L., Fevry, T., Matena, M., Malkan, K., Fiedel, N., Shazeer, N., Lan, Z., Zhou, Y., Li, W., Ding, N., Marcus, J., Roberts, A., and Raffel, C. (2021). Do transformer modifications transfer across implementations and applications? In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5758–5773, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Nielsen, M. A. and Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press. 10th Anniversary Edition.

Northcutt, C. G., Athalye, A., and Mueller, J. W. (2021). Pervasive label errors in test sets destabilize machine learning benchmarks. *arXiv preprint arXiv:2103.14749*. arXiv preprint arXiv:2103.14749.

Planck, M. (1900). Zur theorie des gesetzes der energieverteilung im normalspektrum. *Verhandlungen der Deutschen Physikalischen Gesellschaft*, 2:237–245.

Rath, M. and Date, H. (2023). Quantum data encoding: A comparative analysis of classical-to-quantum mapping techniques and their impact on machine learning accuracy. *arXiv preprint arXiv:2311.10104*. Department of Decision Science, IIM Mumbai.

Recht, B., Roelofs, R., Schmidt, L., and Shankar, V. (2018). Do cifar-10 classifiers generalize to cifar-10? *arXiv preprint arXiv:1806.00451*. arXiv preprint arXiv:1806.00451.

Recht, B., Roelofs, R., Schmidt, L., and Shankar, V. (2019). Do imagenet classifiers generalize to imagenet? In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400. PMLR.

Riquelme, C., Tucker, G., and Snoek, J. (2018). Deep bayesian bandits showdown: An empirical comparison of bayesian deep networks for thompson sampling. *arXiv preprint arXiv:1802.09127*. arXiv preprint arXiv:1802.09127.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.

Schölkopf, B. and Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

Schrödinger, E. (1926). Quantisierung als eigenwertproblem. *Annalen der Physik*, 384:361–376.

Schuld, M., Bergholm, V., Gogolin, C., Izaac, J., and Killoran, N. (2019). Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331.

Schuld, M., Bocharov, A., Svore, K., and Wiebe, N. (2018). Circuit-centric quantum classifiers. *Physical Review A*, 98(3):032331.

Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS 1994)*, pages 124–134. IEEE.

Shor, P. W. (1997). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509.

Smith, S. L., Brock, A., Berrada, L., and De, S. (2023). Convnets match vision transformers at scale. *arXiv preprint arXiv:2310.16764*. arXiv preprint arXiv:2310.16764.